

QFw

The Quantum Framework for HPC

What it is · where it stands · how to get involved

Doug Oucharek · openQSE · July 2026

ORNL is managed by UT-Battelle LLC for the US Department of Energy

Three things, in thirty minutes

1

What QFw is

A quantum execution framework for HPC — and a place to experiment.

2

Where the work stands

Five workstreams in flight: shim, libfabric, a C API, benchmarking, and a local scheduler.

3

How to get involved

QFw is open and working today — its future depends on the community that builds on it.

Quantum is arriving in HPC — as silos



Fragmented interfaces

Each simulator and QPU vendor ships its own SDK, data format, and workflow. No shared shape.



No HPC-native scheduling

Slurm and HPC schedulers don't natively understand a QPU as a resource to reserve and share.



No portability

Applications get hard-wired to one backend — no easy path across simulator ↔ hardware ↔ vendor.

What QFw is



A quantum execution framework for running quantum applications against simulator or hardware services — one workflow, from a laptop to a Slurm cluster.

Distributed runtime

Built on DEFw — the runtime that connects QFw's client, services, and compute across nodes.

QPM services

One Quantum Platform Manager per execution target; it advertises its type and capabilities.

Common schema (qhw)

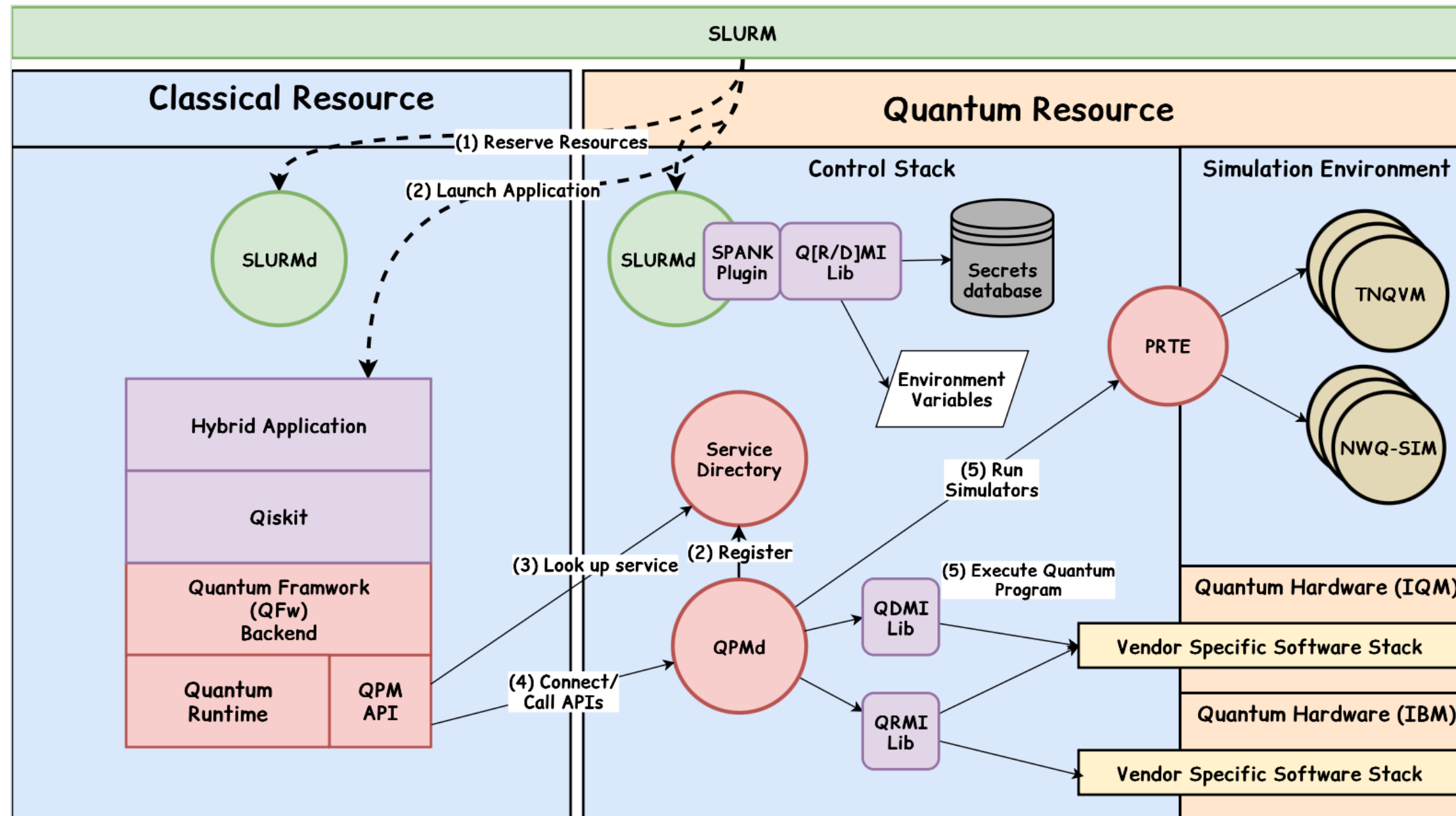
Device and result data are normalized to one schema, so apps never see vendor differences.

One workflow

The same application runs locally, in a Slurm allocation, or in the QFw-SLURM-Cluster container.

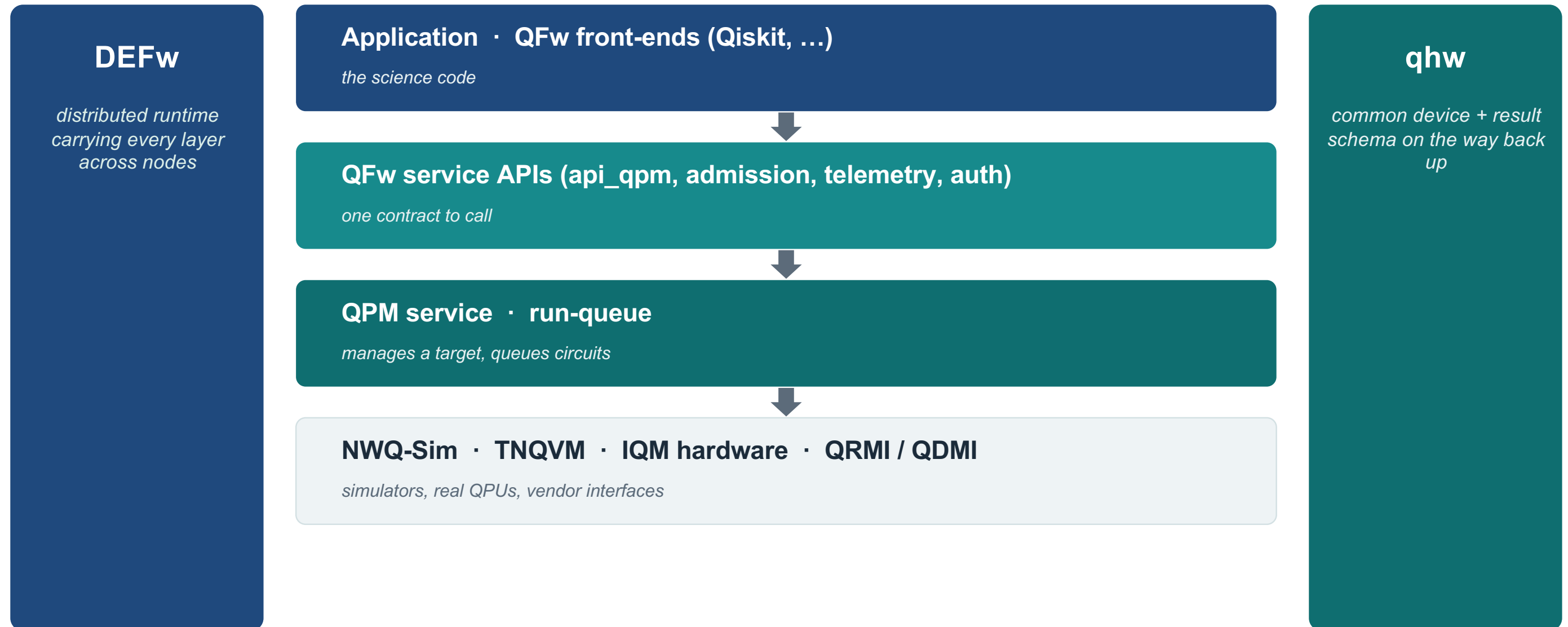
Where QFw fits in the workflow

Classical + quantum resources under one SLURM allocation, brokered by the QPM service (QPMd).



What's inside QFw

The QFw layers behind that workflow — the application never changes as backends come and go.



Four principles behind QFw

1

Vendor-neutral by construction

A contract between users and vendors — not a lowest-common-denominator wrapper. The qhw schema and the shim keep vendor variance out of the application.

2

HPC-native

Quantum resources are scheduled like any other: Slurm, SPANK reservations, GRES. QFw runs where the HPC already runs.

3

Simulator and hardware, one path

NWQ-Sim and TNQVM today, IQM hardware, and more vendors through the shim — same workflow for all of them.

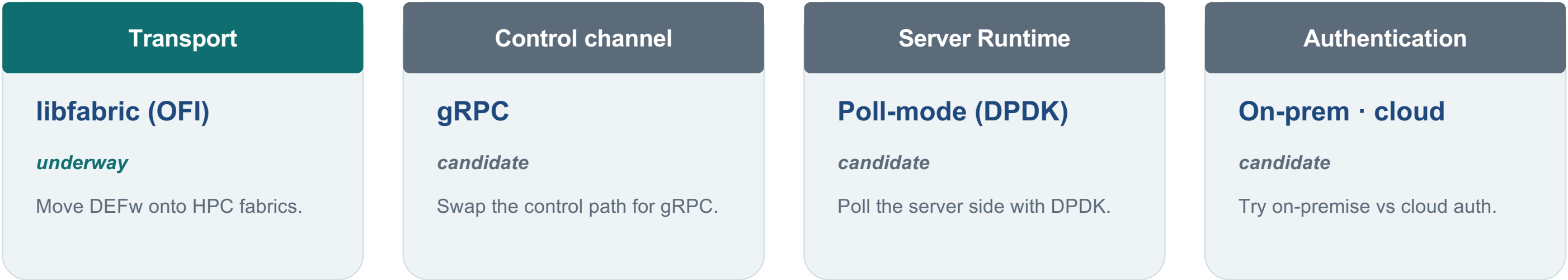
4

Open and collaborative

Developed in the open under openQSE, alongside NWQ-Sim (PNNL), TNQVM (ORNL-QCI), and COMPSs (BSC).

QFw is built for experiments

Those layers aren't just tidy architecture — they make QFw a cheap place to try a bold idea and measure it, without touching the HPC code above.



How an experiment works

Swap one layer, measure the effect with the benchmark, then keep it or drop it. Some experiments become permanent — some don't. That's the point.

PART 2

Where the work stands

Five workstreams in flight — some are experiments, from merged code to an open proposal.

Five workstreams, from merged to proposed

1 Shim layer

MERGED

QRMI / QDMI front-end
One front-end so QFw can drive a QPU through either interface.

2 libfabric

VALIDATED · IN REVIEW

DEFw over OFI
Move DEFw onto HPC fabrics (eg. Slingshot), use RDM for scalability, and RDMA for big payloads.

3 C-based API

DRAFTED

QRI — the neutral contract
The shim's contract as a vendor-neutral C ABI — the spec endgame.

4 Benchmarking

RFC · PROPOSED

testbed + framework overhead
Measure vendors on one testbed, and QFw's own overhead.

5 Local scheduler

MERGED · SIM-TESTED

Credit-based admission + a local scheduler for jobs — repos qhw-admission & qhw-scheduler.

A front-end for QRMI and QDMI

- One front-end so QFw drives a QPU through QRMI (IBM) or QDMI (Munich / MQSS) — or both at once.
- No single interface reaches every QPU. Make the front-end a contract, not a wrapper: the union of what users need.
- Strategy — the libfabric model: ship a working implementation, let it become the de-facto contract, then a vendor-owned spec.



We are here: Implement.

Status

IN QFw MAIN

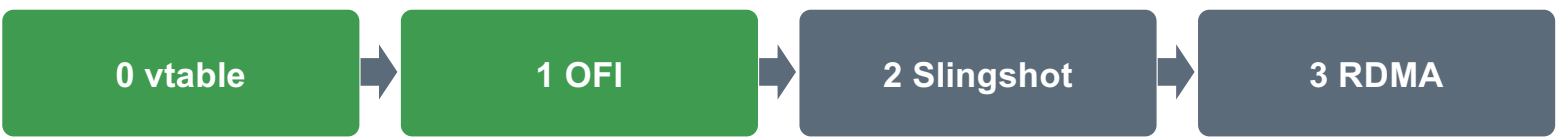
- Device introspection merged — device / coupling / calibration, composable across QDMI + QRMI.
- Circuit execution merged — both the QRMI and QDMI paths, one shared OpenQASM3 pipeline.
- The capability map is real, filled in for IQM q20.
- Next: live-IQM hardware validation of both paths.
- Hope to try the shim on IBM systems too, via QRMI.

Transport for HPC fabrics

The first worked example of an experiment — it may or may not become permanent. QFw is where we find out.

- Move DEFw's client / server communication onto libfabric (OFI).
- Why: run on HPC fabrics like Slingshot (cxi); use RDMA for larger payloads; and scale better with RDM (reliable-datagram) connections.
- How: a transport vtable in DEFw's C layer — TCP for bootstrap and heartbeats, OFI for the RPC data path, per-peer fallback.
- The same vtable is the groundwork for UCX next — so QFw can also run over NVIDIA's networks.
- The Python API never changes.

Four phases



Status

IN REVIEW

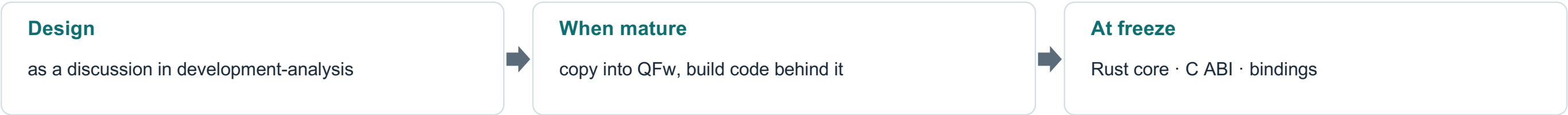
- Phases 0 + 1 complete: vtable refactor, then the full OFI data path.
- Validated in-container over the TCP and shared-memory (sm2) providers, with graceful fallback.
- PRs open on DEFw and QFw, plus the design doc.
- Next: RDMA attachments, then Slingshot (cxi) on real hardware.

QRI — a vendor-neutral C contract

- The shim's contract, expressed as a C ABI: QRI — Quantum Resource Interface.
- Why C: the spec endgame is a C ABI (MPI and libfabric both are). QDMI is C, QRMI is Rust + C — a C core removes Python-binding lag, and one ABI serves C / C++ / Fortran / Julia plus Python.

qri.h	What it covers
<i>the neutral caller API</i> – One set of qri_* calls; unsupported ones return QRI_NOT_IMPLEMENTED — union, not intersection	<i>the qri_* surface</i> – open / close · capabilities · device / coupling / calibration · run_circuit · job timing + metadata

The path



Two kinds of measurement — and how we grade experiments

A · End-to-end run breakdown

Where a complete run spends its time

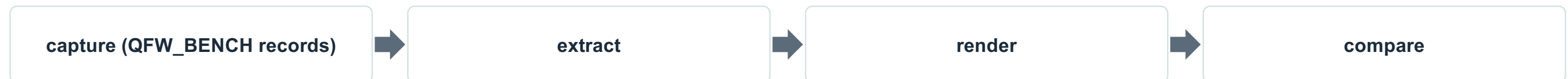
Run a complete workload and break down where all the time goes — QFw, transport, resource API, vendor library, device. Swap any one to compare paths.

B · QFw's own overhead

The framework in the loop

QFw is HPC middleware; its overhead matters, especially for hybrid quantum-classical algorithms that iterate through it.

How it works



- The key to experiments — does libfabric beat TCP? does gRPC help? does poll-mode (DPDK) cut latency? Decide on evidence, not opinion.
- Reuse existing suites (MQT Bench, QED-C, Benchpress); status — an open RFC for community input.

Admission and scheduling, inside QFw

The layer that decides whether QFw takes a job — and what runs next. Two repos.

qhw-admission

credits + authentication

A credit system tells Slurm whether QFw is busy or accepting jobs. Some authentication happens here too.



qhw-scheduler

assign or queue jobs

Once Slurm releases a job, the local scheduler assigns it to a resource — or queues it.

Status

MERGED · SIM-TESTED

- Both repos coded and merged — qhw-admission and qhw-scheduler.
- Credit-based admission gates Slurm; the local scheduler places or queues each job.
- Some authentication is handled in qhw-admission.
- Tested on the simulators.
- Next: validate on real hardware.

PART 3

QFw needs a community

The framework is open and working — its future depends on who experiments with it.

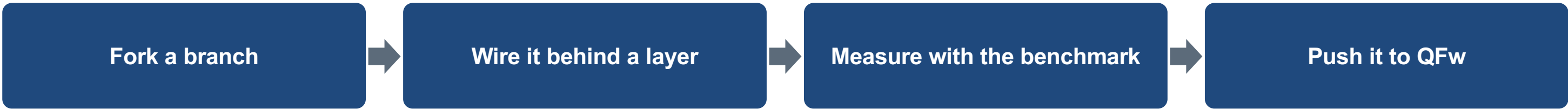
Why participate — QFw is where you experiment

I see my job as QFw's evangelist: make it the easiest place to try a bold idea, measure it honestly, and — when it works — ship it.

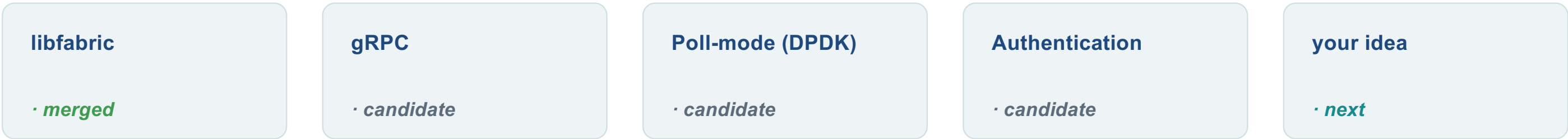
- QFw is a neutral, working testbed — real code, real hardware, one schema, one scheduler. A safe place to try what would be risky in production.
- The layers keep an experiment contained: swap the transport, the control channel, the auth model, or the scheduler policy without breaking the HPC job above.
- The benchmark says whether it actually helped so experiments earn their place on evidence, not opinion.

Bring your experiment — push it as a branch

An idea for a transport, a control channel, a scheduler policy, a data-path trick? The move is the same.



The pipeline so far



Some experiments become permanent. Some don't. Push the branch and let's find out — together.

Where you can plug in

QPU vendors

Add a provider or implement the contract as a branch; shape the QRI headers so the neutral API fits your device.

HPC & fabric folks

Try different fabric APIs (libfabric, UCX), different RPC mechanisms (gRPC), different runtimes (poll-mode), different schedulers, and different authentication approaches.

Quantum-software folks

Bring workloads and benchmark suites so experiments can be judged: tell us what to measure.

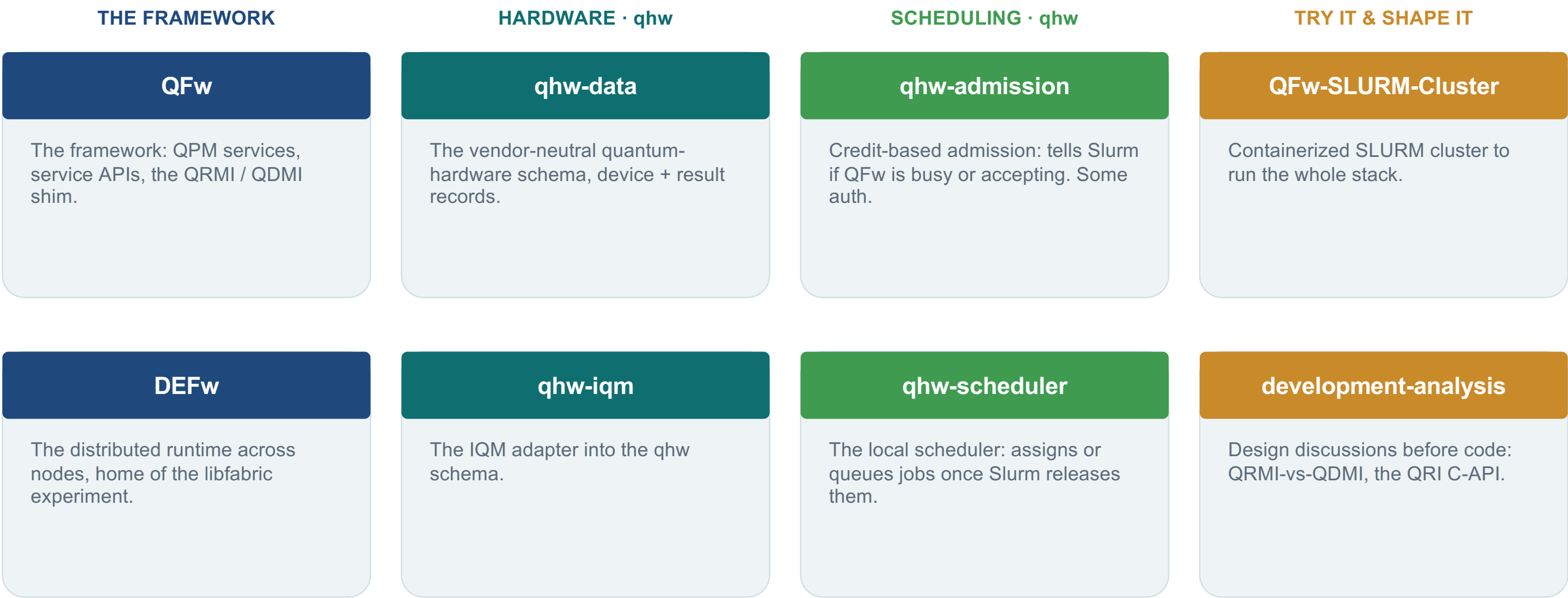
Everyone

Review the design docs, run QFw, and fork a branch to try something. Experiments come in every shape.

It's all under openQSE on GitHub — QFw, DEFw, qhw, development-analysis. Fork it, branch it, come build with us.

Where everything lives — the openQSE repos

Every piece of QFw is a public repo under github.com/openQSE.



QFw is open, working, and built to be experimented on.

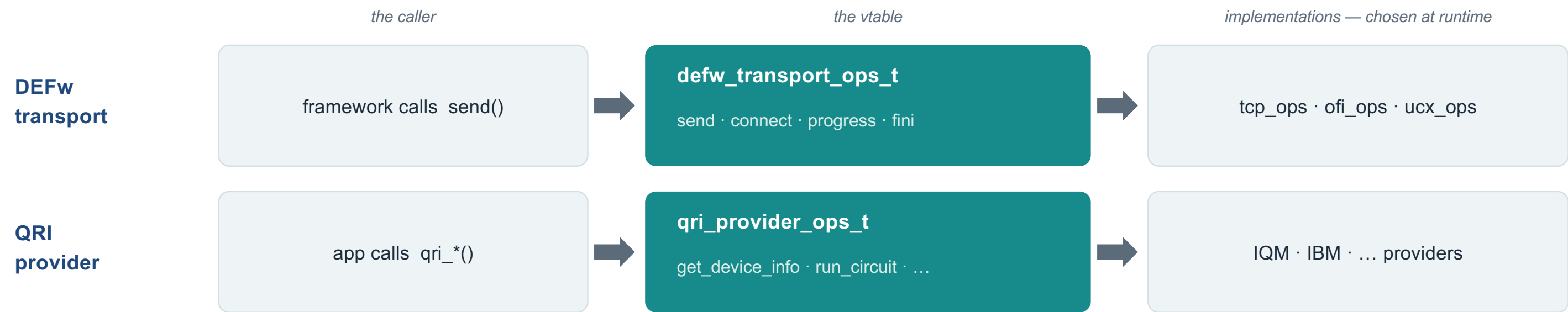
Merged code today · a spec in the making · your next experiment welcome as a branch.

Start with the QFw README and the design docs — github.com/openQSE

Doug Oucharek · [openQSE](#) · evangelist, QFw

What is a “vtable”?

A struct of function pointers — the C way to make a layer pluggable. The same pattern shows up twice in our own stack.



Why we use it

- One call routes through the table to whichever implementation is loaded — that indirection is the whole trick.
- Swap or add a backend with no changes above; an unfilled slot returns QRI_NOT_IMPLEMENTED — union, not intersection.
- Same idiom in DEFw's transport, QRI's providers, and QDMI's driver — deliberate and recurring, not a coincidence.

One shape, everywhere: caller → vtable → pluggable implementation.